JAVA-POWERED PAYROLL ANALYSIS

PAYWISE INTELLIGENCE

Payroll Calculator • Salary Offer Simulator • Smart Audit

FULL APPLICATION PROJECT REPORT

A browser-based payroll application with Java calculations, explainable salary comparisons, rule-based audit insights and a local payroll history journal.

PROJECT DETAILS	VALUE
Prepared by	[Student Name]
Registration / Roll No.	[Registration Number]
Department	Computer Science and Engineering
Institution	[College / University Name]
Project guide	[Guide Name]
Report date	17 October 2026

Implementation: Java 17+ backend • HTML/CSS/JavaScript UI • Dependency-free Java HTTP server

PROJECT SUMMARY

Abstract

Paywise Intelligence is a Java-first payroll analysis application that combines a responsive browser interface with a Java calculation engine and a lightweight HTTP API. The application helps a user model an employee's monthly salary, estimate gross pay and deductions, compare configured old- and new-regime tax estimates, compare two job offers, review rule-based payroll flags, understand differences between saved pay periods, and retain selected payroll snapshots in a local audit journal.

The user interface is divided into focused workspaces: Overview, Payroll Calculator, Tax Advisor, Salary Offer Simulator, Smart Payroll Audit, Salary Change Explainer, Payroll History, and Salary Slip. Java performs salary arithmetic, offer comparison, anomaly checks, and change explanations. The frontend submits inputs to the backend and renders the response; it is not a separate payroll calculation engine.

The backend is implemented with Java 17+ and the JDK HTTP server module, with no third-party runtime dependencies. Payroll history is stored as an append-only, tab-separated local journal. A bundled dependency-free test runner covers salary formulas, configured tax logic, input validation, history handling, insight generation, and offer comparison. At the time of this report, the bundled Java test suite passes 25 of 25 tests.

> **IMPORTANT SCOPE NOTE**
> This is an educational payroll modelling prototype, not a certified payroll system. Tax, HRA, PF and professional-tax logic uses simplified configurable assumptions. Actual statutory liability must be verified independently before operational use.

Project at a glance

Item	Description
Application name	Paywise Intelligence
Application type	Java backend with browser-based frontend
Primary goal	Understand take-home salary and compare modeled compensation
Main differentiator	Offer simulation plus explainable audit and pay-change insights
Persistence	Local append-only TSV journal; optional data directory setting
Runtime	JDK 17 or later; browser for user interface
Verification	25/25 bundled Java tests passed

Keywords: payroll calculation, Java, salary comparison, tax estimation, audit trail, salary change explanation, HTTP API, input validation.

REPORT GUIDE

Contents

Section	Topic
1	Introduction and background
2	Problem statement, motivation and objectives
3	Scope, stakeholders and requirements
4	System architecture and workflow
5	Application modules and user workspaces
6	Calculation methodology and assumptions
7	Salary Offer Simulator: worked example
8	API specification and project structure
9	Testing, verification and results
10	Installation and user guide
11	Data handling, security and deployment
12	Limitations, future work and conclusion
	References and glossary

Abbreviations

Term	Meaning
API	Application Programming Interface
DA	Dearness Allowance
HRA	House Rent Allowance
PF	Provident Fund contribution
PT	Professional Tax
TDS	Tax Deducted at Source; here represented by a modeled monthly tax estimate
80C	Section 80C-related deduction input used by the simplified old-regime model
TSV	Tab-separated values text file
UI	User Interface

HOW TO READ THIS REPORT

The report describes the implementation found in the accompanying source files. Whenever the current model simplifies a real payroll rule, that is identified as an assumption or limitation.

CHAPTER 1

1. Introduction and Background

1.1 Introduction

Payroll calculation turns an employee's salary structure into a clear statement of earnings, deductions and take-home pay. Even a simple structure can involve several dependent values: Dearness Allowance (DA) and House Rent Allowance (HRA) depend on basic pay, PF can depend on basic pay plus DA, Professional Tax (PT) may depend on a configured salary band, and income-tax estimates depend on an annualized salary and selected assumptions.

Paywise Intelligence expands a basic payroll calculator into a small decision-support application. Alongside the calculation page, it offers comparison of two salary offers, configurable tax estimates, rule-based review flags, period-to-period explanations and an audit journal. This combination is useful for a student demonstration because it connects core Java programming to a practical user workflow.

1.2 Background and motivation

Manual calculations or ad-hoc spreadsheets can make it difficult to see why take-home pay differs between two offers or two months. Results may be separated across multiple sheets and assumptions may not be easy to audit. A focused tool can centralize the assumptions, calculate results in one engine, and show the components behind a recommendation.

1.3 Problem statement

The project addresses the need for a lightweight application that can: (a) calculate salary components consistently; (b) estimate deductions and net pay using explicit assumptions; (c) compare alternative salary structures; (d) flag material changes for human review; and (e) retain selected calculation snapshots so later periods can be compared with earlier ones.

1.4 Project objectives

- Implement the payroll calculation rules in Java and keep the browser UI separate from business logic.
- Provide a form-based interface for employee and salary inputs with validation.
- Show gross salary, deductions, net salary and annualized projections.
- Compare modeled old/new tax regimes and investment scenarios using documented assumptions.
- Compare two salary offers using the same calculation engine and report the modeled take-home difference.
- Add explainable rule-based audit checks and salary-change explanations.
- Save selected payroll snapshots with timestamp and model version for traceability.
- Test key calculations and error paths through automated Java tests.

1.5 Success criteria

The project is considered successful when a valid input produces a repeatable calculation, the offer simulator evaluates both offers through the same Java method, invalid inputs are rejected clearly, history can save and retrieve snapshots, pay changes can be explained from stored values, and the supplied automated test suite passes.

CHAPTER 2

2. Scope and Requirements

2.1 Project scope

Paywise targets individual salary calculations and side-by-side modeled offer comparisons. It supports a single running application instance with a local journal file. The user enters employee details and monthly compensation inputs, calculates payroll, optionally saves a snapshot, and can later revisit history to load prior inputs or compare a later period.

Included in current version	Outside current version
Individual payroll calculation and annual projection	Bulk payroll processing for large employee lists
Old/new tax estimate comparison and 80C scenarios	Complete tax computation for every income type and taxpayer case
Two-offer modeled take-home comparison	Automatic valuation of stock, bonuses, benefits and career growth
Rule-based audit flags and change explanations	Machine-learning anomaly detection or proof that an entry is wrong
Local append-only TSV history file	Multi-user database, authentication and access-controlled HR records
Printable/downloadable salary slip view	Certified payroll, statutory filings or bank integrations

2.2 Stakeholders and intended users

- Employees and job applicants: estimate take-home pay and compare salary structures.
- HR/payroll staff in a learning or trial context: review salary components and identify unusual changes for follow-up.
- Students and evaluators: inspect Java modelling, HTTP request handling, testing, and frontend integration.

2.3 Functional requirements

ID	Requirement	Expected behaviour
FR-01	Calculate payroll	Calculate DA, HRA, gross, PF, PT, TDS estimate, deductions and net.
FR-02	Annual projection	Project gross, selected annual tax and net using 12 equal months.
FR-03	Tax advisor	Compare configured regimes and 80C what-if scenarios.
FR-04	Offer simulator	Compute two offers with the same Java payroll engine and compare take-home.
FR-05	Audit insights	Return rule-based flags with severity, title and explanatory text.
FR-06	Change explainer	Compare a current calculation against a saved earlier period for the same ID.
FR-07	History	Save/read snapshots; list recent items and load previous inputs.
FR-08	Salary slip	Display, print or download a salary statement.
FR-09	Validation	Reject blank, malformed, unsupported or negative values with safe messages.

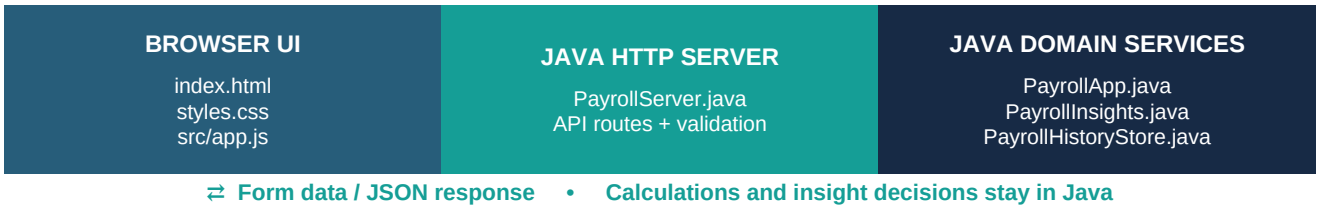
CHAPTER 2-3

3. Quality Requirements and System Architecture

3.1 Non-functional requirements

Quality	Design approach / current boundary
Usability	Focused workspaces reduce a single-page information overload; responsive styles adapt navigation on small screens.
Maintainability	Separate Java classes hold calculation, HTTP, history and insight responsibilities.
Testability	Business logic can be tested without browser automation or a third-party testing framework.
Traceability	Saved snapshots retain input values, modeled results, a save timestamp and model version.
Error handling	Input errors return client-readable messages; malformed form encoding is sanitized.
Security/privacy	The sample app has no authentication or role controls; do not expose it publicly with real payroll data as-is.
Portability	JDK 17+ and a modern browser; launcher scripts are included for Windows and Unix-like systems.

3.2 High-level architecture



3.3 Component responsibilities

Component	Responsibility
Frontend	Collects form values, invokes endpoints, renders estimates and manages view navigation.
PayrollServer	Hosts static assets; validates request method/content type; maps requests into Java records; returns JSON.
PayrollApp	Owns salary/tax data models, salary arithmetic, regime selection and scenario calculations.
PayrollInsights	Evaluates configurable review rules and explains component differences from a prior record.
PayrollHistoryStore	Appends, reads and filters encoded snapshot lines in the local journal file.
PayrollTests	Exercises calculation boundaries, validation and history/insight/offer behaviours.

CHAPTER 3

4. Request Flow and Data Processing

4.1 Main payroll workflow

Step	Stage	Description
1	Input	User enters employee ID, name, department, salary month, basic pay, city type, rent, 80C value, rebate eligibility and PF/PT modes.
2	Validate	Frontend checks required fields and numeric ranges; Java validates the received request again.
3	Calculate	PayrollApp calculates salary components, modeled taxes, chosen regime, monthly net and annualized values.
4	Analyze	PayrollInsights checks the current result and, when available, the latest earlier saved period for the same employee ID.
5	Respond	PayrollServer serializes the result and insights as JSON; the frontend renders cards, breakdowns and tool panels.
6	Save (optional)	Only after the user requests a save does PayrollHistoryStore append a snapshot to the local journal.

4.2 Focused application pages

Workspace	Primary task	Input / output focus
Overview	Navigate to a focused tool and view the latest session summary.	Quick links and current calculated summary.
Payroll Calculator	Calculate employee payroll.	Employee form, earnings, deductions and net pay.
Tax Advisor	Compare tax estimates and investment scenarios.	Regime cards and 80C scenario rows.
Salary Offer Simulator	Compare two offers.	Offer A/B structures, modeled winner and difference.
Smart Payroll Audit	Review flags.	Rule code, severity, title and reason.
Salary Change Explainer	Explain a difference between pay periods.	Deltas for salary components, deductions and take-home.
Payroll History	Review saved snapshots.	Recent records, filters and reuse-previous-input action.
Salary Slip	Prepare a salary statement.	Printable/downloadable current calculated slip.

4.3 State and persistence

The current calculation lives in the active browser session and is refreshed through the Java API. A calculated result is not automatically considered a history record. Users explicitly save snapshots, and the append-only journal records another row for each save. Reusing an old record loads inputs for a new calculation; it does not rewrite the original snapshot.

CHAPTER 4

5. Application Modules - Payroll and Tax

5.1 Payroll Calculator

The Payroll Calculator is the central module. The user enters an employee identifier, name, department and pay month, followed by basic monthly salary, city type, monthly rent, other annual 80C investments and eligibility/configuration selections. The Java engine validates the model and calculates earnings and deductions. Results include monthly and annual views and the selected regime.

- Earnings: Basic, DA, HRA and gross salary.
- Deductions: employee PF, PT, modeled monthly TDS and total deductions.
- Net salary: gross minus the modeled deductions.
- Annual projection: twelve-month projection assumes the monthly pay structure remains constant.

5.2 Tax Advisor

The Tax Advisor shows tax estimates under the configured old and new regimes. The system selects the lower modeled tax for the current input set and displays the alternative for comparison. Scenario rows evaluate defined 80C investment amounts and include the current investment amount when it is not one of the default scenario values.

The new-regime model uses a configured standard deduction of ₹75,000; the old-regime model uses ₹50,000. The old-regime estimate includes a simplified HRA exemption, eligible PF plus other 80C input subject to a configured ₹1,50,000 cap, professional tax and configured slab rates. Final annual tax is modeled with 4% cess and rounded to the nearest ₹10 under the code's chosen convention.

5.3 Configurable assumptions

Parameter	Current model setting	Meaning
DA rate	10% of basic	Allowance assumption used in the prototype.
HRA metro	50% of basic	HRA assumption for metro setting.
HRA non-metro	40% of basic	HRA assumption for non-metro setting.
PF rate	12%	Applied to full Basic + DA or configured capped wage base.
PF ceiling mode	₹15,000/month base	Optional model: min(Basic + DA, ₹15,000) × 12%.
PT Andhra Pradesh	₹0 / ₹150 / ₹200	Configured bands based on modeled monthly gross.
Cess	4%	Configured rate applied in tax estimate.

CONFIGURATION IS NOT A LEGAL OPINION

These values describe the bundled software model. They must be checked against the relevant tax year, state rules, employer policy and employee circumstances before actual payroll processing.

CHAPTER 4

6. Salary Offer Simulator and Intelligence Features

6.1 Salary Offer Simulator

The Salary Offer Simulator is the main decision-support feature. It accepts two independently configurable salary structures and calculates each offer by calling PayrollApp. The comparison reports which offer has the higher modeled monthly take-home and shows monthly and annual differences. Each offer can have its own basic salary, metro/non-metro HRA assumption, rent, 80C input, professional-tax mode and PF contribution model.

The simulator compares the modeled salary components entered by the user. It does not currently calculate total CTC or assign monetary values to employer insurance, bonuses, joining/retention payments, stock/equity, commute costs, job stability, role fit or career progression. The result is one input to a decision, not an automatic recommendation to accept a job.

6.2 Smart Payroll Audit

The audit module applies explainable rules rather than machine learning. It can review component reconciliation, extreme deductions, zero/negative modeled net, manual professional tax, the 80C cap and changes relative to the latest earlier saved snapshot. It labels findings with a severity, code, title and detail. Changes of at least 20% in gross or net pay, and monthly TDS changes of ₹5,000 or more, are configured review prompts.

6.3 Salary Change Explainer

For the same employee ID, the explainer retrieves the most recent saved period that is strictly earlier than the current pay month. It compares Basic, DA, HRA, gross salary, PF, PT, TDS, total deductions and net pay. Explanation text describes numeric differences and may suggest checking for a raise, unpaid leave, arrears or changed input assumptions. It does not infer a cause that is not recorded.

6.4 Payroll History and Audit Trail

PayrollHistoryStore writes a versioned snapshot with a unique record ID and timestamp to a local append-only TSV file. Selected input fields and calculated outputs are encoded to preserve tab-separated row structure. The encoding is field separation only; it is not encryption. A snapshot can be loaded to reuse its inputs, while the original row remains in the journal.

6.5 Salary Slip and Overview

The Salary Slip workspace presents the latest calculation as a statement that can be printed or downloaded. Overview provides navigation and the latest session summary. The interface separates tools to reduce clutter while keeping a consistent design system.

CHAPTER 5

7. Calculation Methodology and Assumptions

All core calculation steps are implemented in PayrollApp.java. Money components are rounded to two decimal places at relevant calculation boundaries. Tax calculations preserve slab precision until the configured cess/relief treatment is applied, and the current model rounds taxable income and final annual tax to the nearest ₹10.

Item	Current formula / rule	Notes
DA	Basic × 10%	Configurable assumption.
HRA	Basic × 50% (metro) or 40% (non-metro)	City type is a user input.
Gross	Basic + DA + HRA	Sum of modeled earnings.
PF - full basis	(Basic + DA) × 12%	Simplified employee contribution model.
PF - ceiling basis	min(Basic + DA, ₹15,000) × 12%	Optional simplified wage-ceiling mode.
PT - AP schedule	Gross ≤ ₹15,000: ₹0; ≤ ₹20,000: ₹150; otherwise ₹200	State-specific simplified configuration.
New taxable income	max(0, annual gross − ₹75,000)	Configured standard deduction.
Old HRA exemption	max(0, min(actual HRA, rent − 10% of salary base, metro/non-metro cap))	Simplified “least of” model.
Old 80C deduction	min(₹150,000, annual PF + other 80C input)	Configured cap.
Monthly TDS	Chosen annual modelled tax ÷ 12	Projected evenly each month.
Net salary	Gross − PF − PT − monthly TDS	Does not represent full legal payslip in every circumstance.

7.1 Tax-regime selection

The model computes both regimes from the configured assumptions, then chooses the regime with the lower modeled annual tax. New-regime rebate is only applied when the user indicates eligibility. The old-regime rebate threshold is similarly conditional. Marginal relief is modeled in the new-regime path, and annual tax is rounded according to the software’s documented convention. This is not a full recreation of all Indian tax rules.

7.2 Validation rules

- Employee ID, name, department, valid YYYY-MM pay period and calculation modes are required.
- Salary, rent, investments and manual PT must be finite, non-negative numbers within configured supported limits.
- Invalid PF or PT modes, unsupported city/eligibility values, malformed requests and professional tax above gross are rejected.
- Input validation occurs in the frontend for usability and again in Java for the authoritative calculation path.

CHAPTER 5

8. Worked Example: Comparing Two Job Offers

This scenario shows how the Salary Offer Simulator uses the same Java calculation engine for both offers. The figures represent the configured model and not statutory tax advice.

8.1 Test inputs

Input	Offer A: TechNova	Offer B: CloudPeak
Monthly basic salary	₹50,000	₹60,000
City setting	Metro	Non-metro
Monthly rent	₹20,000	₹15,000
Other annual 80C input	₹50,000	₹40,000
PF basis	12% of full Basic + DA	12% up to ₹15,000 wage base
Professional tax	Configured AP schedule	Configured AP schedule
Rebate eligibility input	Eligible	Eligible

8.2 Monthly output comparison

Component	TechNova (A)	CloudPeak (B)
Basic	₹50,000	₹60,000
DA	₹5,000	₹6,000
HRA	₹25,000	₹24,000
Gross salary	₹80,000	₹90,000
PF	₹6,600	₹1,800
Professional tax	₹200	₹200
Modeled monthly TDS	₹0	₹0
Total modeled deductions	₹6,800	₹2,000
Modeled net salary/month	₹73,200	₹88,000
Modeled annual net	₹8,78,400	₹10,56,000

SIMULATOR RESULT

CloudPeak has the higher modeled take-home by ₹14,800 per month, equal to ₹1,77,600 over 12 projected months. This is primarily influenced by the higher modeled gross salary and the selected PF contribution basis. Review both employers' actual PF policy and total benefits before making a decision.

8.3 Manual calculation check

TechNova: DA = ₹50,000 × 10% = ₹5,000; HRA = ₹50,000 × 50% = ₹25,000; gross = ₹80,000; PF = (₹50,000 + ₹5,000) × 12% = ₹6,600; PT = ₹200; modeled net = ₹80,000 − ₹6,600 − ₹200 = ₹73,200.

CloudPeak: DA = ₹60,000 × 10% = ₹6,000; HRA = ₹60,000 × 40% = ₹24,000; gross = ₹90,000; PF = min(₹66,000, ₹15,000) × 12% = ₹1,800; PT = ₹200; modeled net = ₹90,000 − ₹1,800 − ₹200 = ₹88,000.

CHAPTER 6

9. API Design and Project Structure

The Java HTTP server exposes JSON-based endpoints to the browser UI. POST routes accept application/x-www-form-urlencoded input. The server checks content type and request size, parses fields, builds Java domain records, invokes the calculation or storage method, and returns a JSON response with an appropriate HTTP status.

Method	Endpoint	Purpose / behaviour
GET	/api/health	Reports status, application name and Java engine status.
POST	/api/payroll	Calculates payroll and attaches rule-based insights against earlier saved history when available.
POST	/api/offer-compare	Calculates Offer A and Offer B through PayrollApp and returns the modeled winner and differences.
GET	/api/history	Lists recent snapshots; optional employeeId and limit query parameters filter the results.
POST	/api/history	Recalculates submitted employee input and appends the resulting snapshot to the journal.
GET	/api/audit	Alias for viewing saved audit/history entries.

9.1 Source-code organization

File / folder	Responsibility
PayrollApp.java	Employee, tax, scenario and result records; configuration and payroll algorithms.
PayrollServer.java	HTTP routes, request parsing, validation, JSON output and static asset serving.
PayrollInsights.java	Rule-based audit findings and salary-change explanations.
PayrollHistoryStore.java	Snapshot model, line encoding, append/read/filter and prior-period lookup.
PayrollTests.java	Dependency-free Java automated test runner.
index.html	Semantic page structure and eight focused workspaces.
styles.css	Responsive visual style and tool-page layouts.
src/app.js	Frontend validation, navigation, fetch calls and result rendering.
run.bat / run.sh	Compile, test and launch scripts.
Dockerfile / render.yaml	Deployment examples for a Java-capable host/container.

9.2 API error behaviour

The API rejects unsupported HTTP methods and content types, oversized submitted forms, malformed URL-encoded data and invalid payroll fields. User-facing error messages avoid returning raw URL-decoder exceptions. Unexpected server failures return a generic response to the client while diagnostic detail may be written to the server log.

CHAPTER 7

10. Testing and Verification

The project includes PayrollTests.java, a dependency-free test runner. It can be executed after compilation with `java -ea -cp out PayrollTests`. In the verification run for this report, all 25 bundled tests passed.

> **TEST RESULT**
> 25 tests passed • 0 tests failed. This verifies the covered Java logic; it does not replace statutory payroll validation, multi-user security testing, load testing, or a full visual audit across all browsers.

10.1 Test coverage summary

Test area	Examples covered	Status
Salary arithmetic	Basic, DA, HRA, gross, monthly/annual values	PASS
PF modes	Full Basic + DA and ₹15,000 wage ceiling	PASS
Professional tax	AP threshold, manual amount and no-PT selection	PASS
Tax model	Slab boundaries, rounding to ₹10, rebate eligibility, marginal relief	PASS
Old-regime calculation	80C cap and zero-rent HRA exemption	PASS
Input validation	Negative salary, invalid modes, month format and unsupported values	PASS
Request parsing	Malformed URL-encoded form is sanitized	PASS
Scenario modelling	Current investment included in what-if scenarios	PASS
History	Write/read snapshots and find only an earlier period	PASS
Insight logic	Material gross change can be flagged with explanation	PASS
Offer simulator	Both offers use the common Java calculation model	PASS

10.2 Representative test cases

Input / condition	Expected behaviour	Observed test status
Basic salary = ₹0	No negative outputs; valid non-negative values accepted.	PASS
PF mode = wage-ceiling	PF base is capped at configured ₹15,000.	PASS
Professional tax = AP schedule	Configured salary threshold determines PT.	PASS
Negative basic salary	Input is rejected by validation.	PASS
Malformed id=%ZZ form	Client receives a safe invalid-form message.	PASS
Saved earlier salary month	History lookup chooses earlier matching employee period.	PASS
Two different offer inputs	Both are calculated and a modeled winner/difference is returned.	PASS

CHAPTER 8

11. Installation and User Guide

11.1 Prerequisites

- JDK 17 or later installed; `java -version` and `javac -version` should work in a terminal.
- A modern browser such as Chrome, Edge, Firefox or Safari.
- Project files extracted into one directory. Do not open index.html directly with file://; use the Java server so API calls work.

11.2 Run on Windows

1. Extract or clone the project folder.
2. Double-click `run.bat`, or open Command Prompt in the project directory and run `run.bat`.
3. The script compiles the source and runs the Java test suite. The server starts only if the tests pass.
4. Open `http://localhost:8080` in a browser and keep the terminal open while using the application.
5. Press Ctrl+C in the terminal to stop the server.

11.3 Run on macOS / Linux

Open a terminal in the project directory and run:

```
./run.sh
```

To compile and run tests manually:

```
mkdir -p out
javac -encoding UTF-8 -Xlint:all -d out PayrollApp.java PayrollServer.java PayrollHistoryStore.java
PayrollInsights.java PayrollTests.java
java -ea -cp out PayrollTests
java -cp out PayrollServer
```

11.4 Suggested user workflow

- Start in Overview and choose the relevant workspace.
- For payroll, complete employee inputs and calculate; inspect earnings, deductions, tax assumptions and net pay.
- For offer comparison, enter both salary structures and compare modeled monthly/annual take-home.
- For pay-change analysis, save one period first, then calculate a later period for the same employee ID.
- Save only the records you intend to retain; review the history view before reusing old inputs.
- Print or download the salary slip only after checking employee and calculation inputs.

CHAPTER 9

12. Data Handling, Security and Deployment

12.1 Local history storage

PayrollHistoryStore writes selected snapshots to `data/payroll-history.tsv` by default. The directory can be configured through `PAYWISE_DATA_DIR`. Each snapshot records a unique identifier, saved timestamp, model version, employee key, pay month, entered assumptions and calculated outputs. The file is append-only at application level; the current interface does not provide authenticated deletion, update or retention-management controls.

12.2 Privacy and security boundary

- The standard local configuration binds to 127.0.0.1, making the service accessible from the local machine by default.
- The history journal contains sensitive salary and employee details. Never commit the generated `data/payroll-history.tsv` to a public repository.
- Base64 encoding preserves field separation; it is not encryption and does not protect stored records from anyone with file access.
- No user authentication, role-based authorization, session management, database encryption or data-retention policy is implemented in this prototype.
- Before public deployment with real employee data, add authentication, HTTPS, authorization, protected storage, backups, retention rules, monitoring and a security/privacy review.

12.3 Online deployment

The repository contains a Dockerfile and a Render Blueprint example as starting points for a Java-capable host. The backend reads the port from the `PORT` environment variable and host binding from `HOST`. A container deployment typically needs `HOST=0.0.0.0`. If saved records must survive a restart or redeploy, attach persistent storage and set `PAYWISE_DATA_DIR` to the mounted directory.

> **GITHUB PAGES NOTE**
>
> GitHub can host the source repository, but GitHub Pages cannot execute this Java HTTP backend. To serve a working application online, deploy the Java service to a Java-capable host/container and use persistent storage if history is required.

12.4 Operational considerations

Keep application source, generated class files and user data separated. Back up the journal where appropriate and control file permissions. Test the deployed build after changes; an online service without a persistent disk can lose local file history during redeployment. Do not treat a successful page load as proof that payroll APIs and storage are working; check `/api/health` and carry out a harmless calculation test.

CHAPTER 10

13. Limitations and Future Enhancements

13.1 Current limitations

Limitation	Impact	Suggested next step
Simplified tax rules	May not represent an individual's actual tax liability.	Review all rules by tax year; add verified tests and qualified review.
Java double for money	Binary floating-point can create small decimal differences.	Migrate money arithmetic to BigDecimal and an explicit rounding policy.
No authentication	No separation of one user's payroll data from another.	Introduce authentication and role-based access before a public deployment.
Flat-file history	Limited querying, concurrency control and operations management.	Move history to a relational database with migrations and backups.
Offer scope	Bonuses, equity, employer benefits and non-salary factors are not valued.	Add total-compensation inputs and transparent optional valuation methods.
Rule-based flags	Flags indicate review candidates, not confirmed payroll errors.	Allow reviewer notes, resolution state, rule thresholds and audit evidence.
Manual/annual assumptions	Annual projection assumes 12 similar months and no variable pay.	Add bonus, leave, arrears and month-specific TDS projection features.

13.2 Future enhancements

- Use BigDecimal for currency and add currency-safe formatting/rounding tests.
- Persist data in PostgreSQL or another relational database with schema migrations.
- Implement login, roles, secure sessions and audit access controls.
- Add CSV import/export and multi-employee batch payroll processing.
- Extend offer comparison to base pay, variable bonus, employer PF, insurance, joining bonus, stock/equity, commute and benefits, keeping assumptions explicit.
- Allow authorized reviewers to classify audit findings as accepted, resolved or requiring follow-up, with comments.
- Add browser-based end-to-end tests, accessibility testing, responsive screenshots and deployment health monitoring.
- Version tax and payroll rules by financial/tax year with sources and effective dates.

13.3 Conclusion

Paywise Intelligence demonstrates how a basic Java salary calculator can be developed into a more useful payroll analysis application. Its focused workspaces combine core salary calculations, modeled tax comparison, job-offer simulation, explainable review flags, salary-change analysis and snapshot history. Keeping calculations and recommendation logic in Java makes the core engine testable and separates business decisions from UI rendering.

The project is suitable as an educational software-engineering prototype and a foundation for further work. Its most valuable next steps are stronger currency arithmetic, verified year-specific payroll rules, a database-backed data model and proper access controls before handling real employee information.

APPENDIX

References and Glossary

References

1. Paywise Intelligence source package: PayrollApp.java, PayrollServer.java, PayrollInsights.java, PayrollHistoryStore.java, PayrollTests.java, index.html, styles.css and src/app.js.

2. Paywise Intelligence README.md included in the repository. This is the authoritative project-specific description of supported modules, routes, run instructions and limitations.

3. Oracle, Java Platform Standard Edition 17 API documentation - HTTP server module: https://docs.oracle.com/en/java/javase/17/docs/api/jdk.httpserver/module-summary.html

4. Income Tax Department of India, e-Filing portal for current official tax information: https://www.incometax.gov.in/ (consult the rules applicable to the payroll/tax year before using results).

5. GitHub Pages documentation: https://docs.github.com/en/pages (for source-site hosting; the Java API must be hosted separately).

Glossary

Term	Definition in this project
Modeled tax	An estimate produced by the application’s configured rules; not a certified tax determination.
Salary structure	Basic salary and the allowance/deduction assumptions used in a calculation.
Salary offer comparison	Side-by-side calculation of two modeled salary structures under entered assumptions.
Audit flag	A rule-generated prompt to investigate an input or change; not proof that it is wrong.
Snapshot	A saved copy of employee inputs and calculated outputs for one pay month.
Model version	Identifier recorded with history to help indicate which rule set produced a snapshot.
Append-only journal	A file to which new records are appended rather than replacing prior records through the application.

END OF REPORT

Paywise Intelligence — Java-powered payroll modelling, salary-offer comparison and explainable payroll review.